VGB Invoice Subsystem

Design Document

Oscar Hanson
ohanson5@huskers.unl.edu
Ermias Wolde
ewolde2@huskers.unl.edu
University of Nebraska—Lincoln

Spring 2025
Version 5.0

Abstract

This document describes a model used to create an invoicing subsystem for the Very Good Building & Development Company (VGB). The document is a collection of all the design and implementation processes accumulated throughout the scope of the project. It details the requirements, processes, objects, testing strategies, and changes used to develop an invoicing subsystem that is able to interface with other subsystems developed by VGB.

Revision History

c

Version	Description of Change(s)	Author(s)	Date
1.0	Brainstormed Design purpose	Oscar Hanson, Ermias Wolde	Feb 14, 2025
2.0	Added Identifiable and Person class	Oscar Hanson, Ermias Wolde	Feb 26, 2025
3.0	Added subclasses via UMLs	Oscar Hanson, Ermias Wolde	Mar 28, 2025
4.0	Added export/report tools	Oscar Hanson, Ermias Wolde	Apr 4, 2025
5.0	Included test classes and factory classes	Oscar Hanson, Ermais Wolde	Apr 25, 2025
6.0	Added custom data structure LinkedList	Oscar Hanson, Ermais Wolde	Apr 25, 2025

Contents

1 Introduction **3**
- 1.1 Purpose of this Document . . . 4
- 1.2 Scope of the Project . . . 4
- 1.3 Definitions, Acronyms, Abbreviations . . . 4
  - 1.3.1 Definitions . . . 4
  - 1.3.2 Abbreviations & Acronyms . . . 5

2 Overall Design Description **5**

3 Detailed Component Description **6**
- 3.1 Database Design . . . 6
  - 3.1.1 Component Testing Strategy . . . 8
- 3.2 Class/Entity Model . . . 9
  - 3.2.1 Component Testing Strategy . . . 19
- 3.3 Database Interface . . . 20
  - 3.3.1 Component Testing Strategy . . . 21
- 3.4 Design & Integration of a Sorted List Data Structure . . . 21
  - 3.4.1 Component Testing Strategy . . . 22

4 Changes & Refactoring **22**

5 References **22**

Bibliography **22**

1 Introduction

The Very Good Building & Development Company (VGB) is expanding and needs to modernize. The company had previously kept track of all its data in spreadsheets and physical paper. However, VGB's business is involved in all aspects of construction: acting as a general contractor creating and awarding subcontracts, selling, leasing, and renting construction equipment and building materials related to construction. As such, the company seeks to update all business operations with newly developed systems for inventory, marketing, delivery, invoicing, and sales. This design document, specifically, focuses on the development of the invoicing subsystem.

The invoicing subsystem is responsible for managing invoices, calculating taxes including handling exemptions, organizing and storing billing and transaction data within a database-backed system, and producing detailed invoice and financial reports for the business. It is capable of parsing, handling, storing, and serializing data related to company invoices. Specifically, the subsystem provides support for parsing and processing data about people, companies, and company items (equipment, materials, and contracts), including Universally Unique IDs (UUIDs), names, addresses, company information, equipment, prices, tax rates, and exemptions. It ensures this information is persistently stored in a database, allowing it to be reliably updated, retrieved, and managed over time. Additionally, the subsystem supports the conversion of invoicing data into generalized formats (XML, JSON) to facilitate integration with other subsystems being developed across the company.

The invoicing subsystem not only generates and tracks invoices but also manages billing data for people, companies, and items. It performs tax calculations with built-in logic for exemptions (e.g., nonprofit organizations or tax-exempt items) and generates detailed financial and expenditure reports. By automating these critical processes, the subsystem improves the efficiency, accuracy, transparency, and scalability of VGB's invoicing operations, aligning closely with the company's broader modernization objectives.

Specific business and tax rules include:

- Every transaction (sale, lease, rental, or contract) must generate a unique invoice containing a UUID.
- Each invoice must include customer information, itemized charges, a breakdown of applicable state and local taxes, and a total amount.
- Tax exemptions apply to government entities and registered nonprofits, verified via exemption certificates.
- Payment terms are Net-30 by default unless otherwise specified.
- Invoices unpaid after 30 days will trigger automatic reminder emails; after 60 days, a 2% monthly late fee is applied.
- Rounding rules ensure all monetary amounts are rounded to the nearest cent.

1.1 Purpose of this Document

This document provides the technical design specification for the invoicing subsystem of the Very Good Building & Development Company (VGB), supporting the transition from manual record-keeping to a database-backed system. It describes the overall system structure, defines the core business rules and tax calculation procedures, and outlines the major components of the subsystem, including invoicing, billing, payment tracking, and financial reporting features. The document also provides a foundation for the detailed design and development of the invoicing subsystem, ensuring that the implementation is consistent with the company's operational needs and modernization goals. The design process is presented step-by-step to allow accurate reproduction and traceability in future development work, following standards aligned with IEEE 1016.

1.2 Scope of the Project

This project is responsible for handling, storing, validating, and packaging data specifically related to invoices, billing, payments, and financial reporting for VGB's operations. The invoicing subsystem manages the generation of invoices for transactions involving company contracts, equipment, materials, and services. It tracks amounts owed and amounts to be received by the company, calculates applicable taxes (including exemptions), and supports the production of detailed financial reports.

The invoicing subsystem does **not** manage inventory operations, marketing activities, delivery logistics, or general sales processes. These areas are handled by separate subsystems that may interface with the invoicing system by providing relevant financial data but are outside the scope of this project

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

API A way to allow different pieces of software to interact with one another. Most often, this occurs by a piece of software providing a set of rules or a contract that can be used by other pieces of software.

Business Amortization The process of gradually writing off the cost of intangible assets (e.g. patents, trademark, goodwill) over the life of the item. In other words, averaging out the revenue of an asset over its useful life.

Invoice A formal document that is issued by a seller to a buyer that details the goods or services provided and the amount owed. Usually serves as a request for payment.

Serialization The process of formatting object data in a way that can be sent and "deserialized" into a representation of the same object. Common formats include

JSON and XML.

JavaScript Object Notation A commonly used format for data serialization that is human-readable.

Extensible Markup Language A commonly used format for data serialization that is human-readable.

Universally Unique Identification A string or code that is used to uniquely identify companies, people, equipment, materials, or contracts.

Comma Separated Values (File) A plain-text file that stores information in field(s) defined on the first line. An example of a first line would be "name, age, height", and the following lines would specify information for one "thing" that contains all or some of the above information in order, such as "Jacob, 23, 5ft 9in" or something similar.

Java Database Connectivity A common and standardly used API

1.3.2 Abbreviations & Acronyms

API Application Programming Interface

VGB Very Good Building & Development Company

UUID Universally Unique Identification

JSON JavaScript Object Notation

XML Extensible Markup Language

IEEE Institute of Electrical and Electronics Engineers

CSV Comma Separated Values (File)

JDBC Java Database Connectivity

2 Overall Design Description

The invoicing subsystem stores all invoicing-related data in a database, enabling the retrieval, updating, deleting, and adding records as needed. This persistent storage ensures long-term access to consistent data across sessions. The database uses MySQL and interfaces with Java objects via Java Database Connectivity (JDBC). All database operations are logged and monitored for security and maintenance purposes.

Objects in the system—such as Person, Company, Material, Equipment, and Contract—are created by reading data either from the database or from files. The system supports basic CSV file parsing and uses factory classes to load objects from the

database. The Address object is used as a component of the Company class, and all invoiceable items inherit from the abstract Item class, which is extended by Material, Equipment, and Contract.

When other subsystems require access to invoice data, the invoicing subsystem provides serialized outputs in JSON (via the ToJSON class) or XML (via the ToXML class) formats. These formats enable consistent data exchange and allow seamless de-serialization by external modules.

3 Detailed Component Description

This section provides an overview of the implementation details for several major components of the invoicing subsystem. It covers the design of the database and associated testing strategies, the development of the class and entity models, the database interface and its testing approach, and the design and integration of a sorted list data structure along with its testing methodology. Each subsection describes the structure and behavior of a specific component, the relevant data it manages or processes, and the strategies used to verify its correctness and reliability within the overall system.

3.1 Database Design

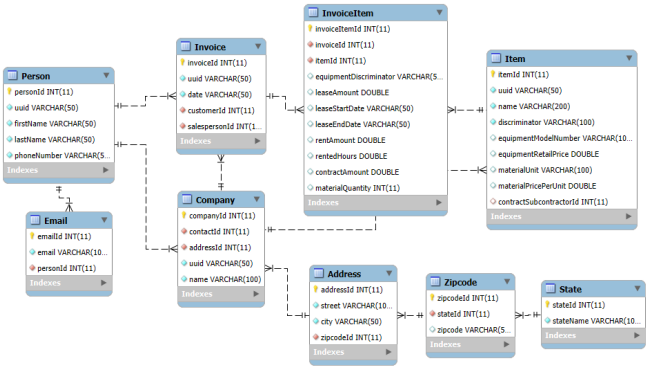

Figure 1: An ER diagram of the database.

Key Entities and Their Purposes

- **Person**

-Stores data on individuals (e.g., UUID, name, address, contact details) involved in invoicing, either as clients or internal contacts. This is essential to associate personal billing data with invoices and ensure accurate communication with stakeholders.

- **Company**

-Maintains information about partner companies, contractors, subcontractors, and suppliers. Given the business model of VGB, it is vital to track the invoicing data associated with various corporate entities, ensuring precise reporting on expenditures and income in all operations.

- **Address**

-Contains information on the physical address of the respective company, including the street, city, state (transitively) and zip code.

- **Zipcode**

-Represents the zip code of an address, while also linking the state to the address.

- **State**

-Represents a state (of the United States). States are normalized (there are only 50) in order to maintain good data integrity and make it easy to change.

- **Item**

Captures details about company items, including equipment, materials, and contracts (with attributes such as UUID, description, price, and tax details). Centralizing item information facilitates dynamic pricing and inventory management, and ensures that all invoicing details related to tangible and service items are consistent and easily updateable.

- **Invoice**

- As the core record for each invoice, including invoice IDs, dates, and references to the relevant People or Companies. The *Invoices* table aggregates all billing data and acts as the central node to track financial transactions, aligning with the subsystem's goal of automating and streamlining invoice management.

- **InvoiceItem**

-Manages the many-to-many relationship between Invoices and Items. It records the quantity, unit price, and any additional charges for each item included in an invoice. This design minimizes redundancy by separating item details from the invoice record while allowing a single invoice to list multiple items, which is crucial for generating detailed and accurate financial reports.

Design Considerations

- **Data Integrity and Normalization:**

The design is normalized (up to 3NF) to minimize data redundancy. For example, item details are stored once in the *Items* table, and associations to invoices are maintained via *InvoiceItems*, ensuring consistent and reliable data updates.

- **Scalability and Flexibility:**

The expansion of VGB into multiple business areas requires a design that is flexible enough to accommodate additional features (such as integration with inventory, marketing, and delivery systems) without major restructuring.

• **Performance Optimization:**

Indexing key fields (like UUIDs, invoice IDs, and dates) increases efficiency and ensures that the system can efficiently handle a high volume of transactions and generate timely financial reports, which is essential for the operational needs of VGB.

• **Alignment with Business Operations:**

Each entity directly supports the operational needs of VGB:

• *People* and *Companies* capture the diverse stakeholders involved in the invoicing.

• *Items* provide a unified source of pricing and inventory data.

• *Invoices* and *InvoiceItems* record the detailed financial transactions.

• *Payments* track monetary flows, crucial for expenditure and income reporting.

3.1.1 Component Testing Strategy

An external testing environment provided by the client was used to verify the integrity and basic functionality of the database. In addition to external validation, approximately 20 internal database tests were developed and executed. These tests covered the creation, retrieval, updating, and deletion of records across key entities, including People, Companies, Addresses, Items, Invoices, InvoiceItems, and Payments.

Testing was conducted on a reduced-scale version of the database that maintained the same structure, allowing for faster iteration and controlled debugging. The tests confirmed that foreign key constraints were properly enforced, cascading deletes behaved as expected, and the schema could accommodate realistic business data without errors.

Initial testing uncovered minor issues, such as missing foreign key constraints and incorrect data type assignments in some tables. These issues were promptly corrected, improving overall consistency and stability. No major redesigns were required. The testing process confirmed that the database met the functional and performance requirements necessary for integration with the invoicing subsystem.

3.2 Class/Entity Model

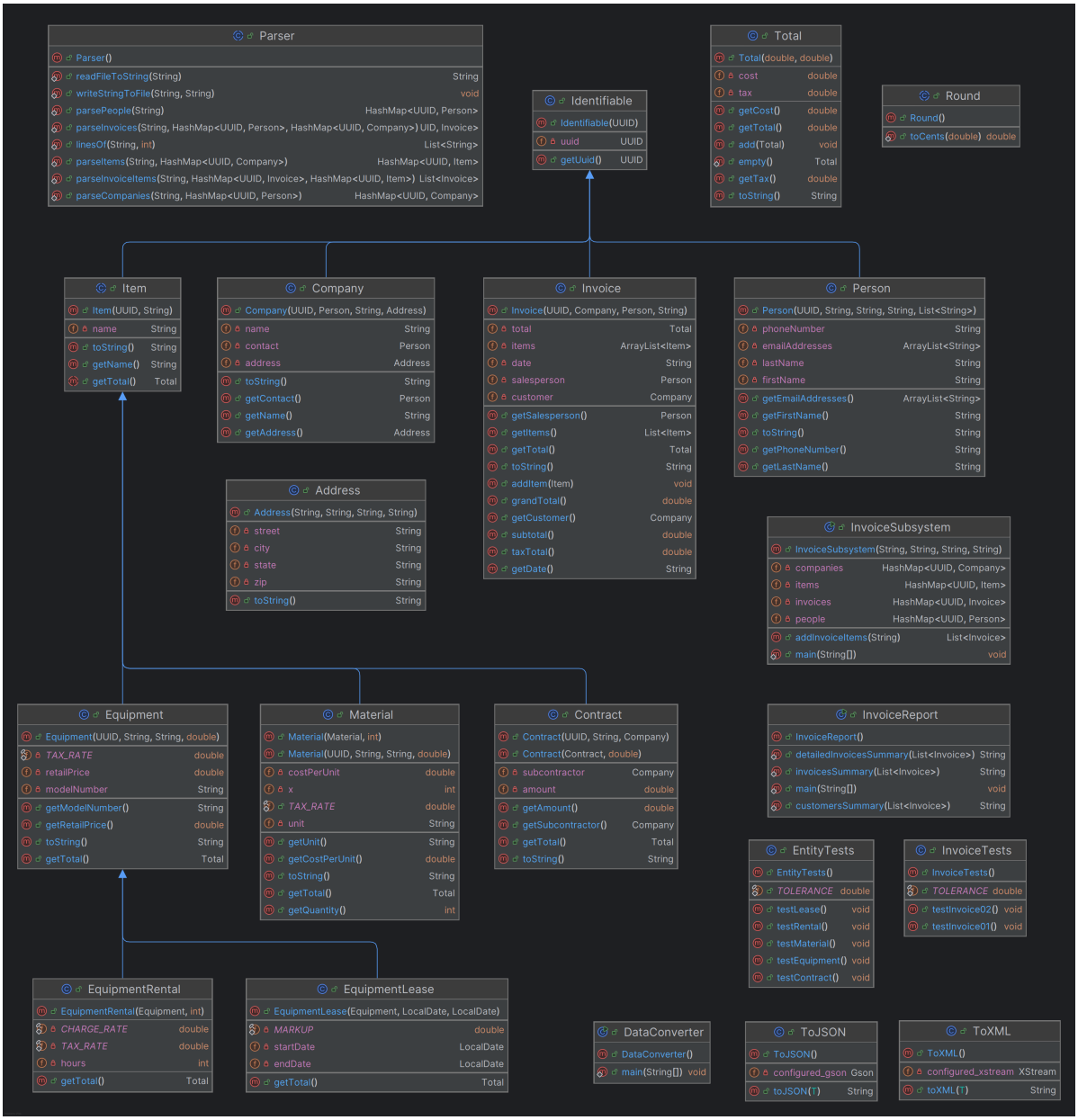

Figure 2: A UML diagram of the invoice subsystem.

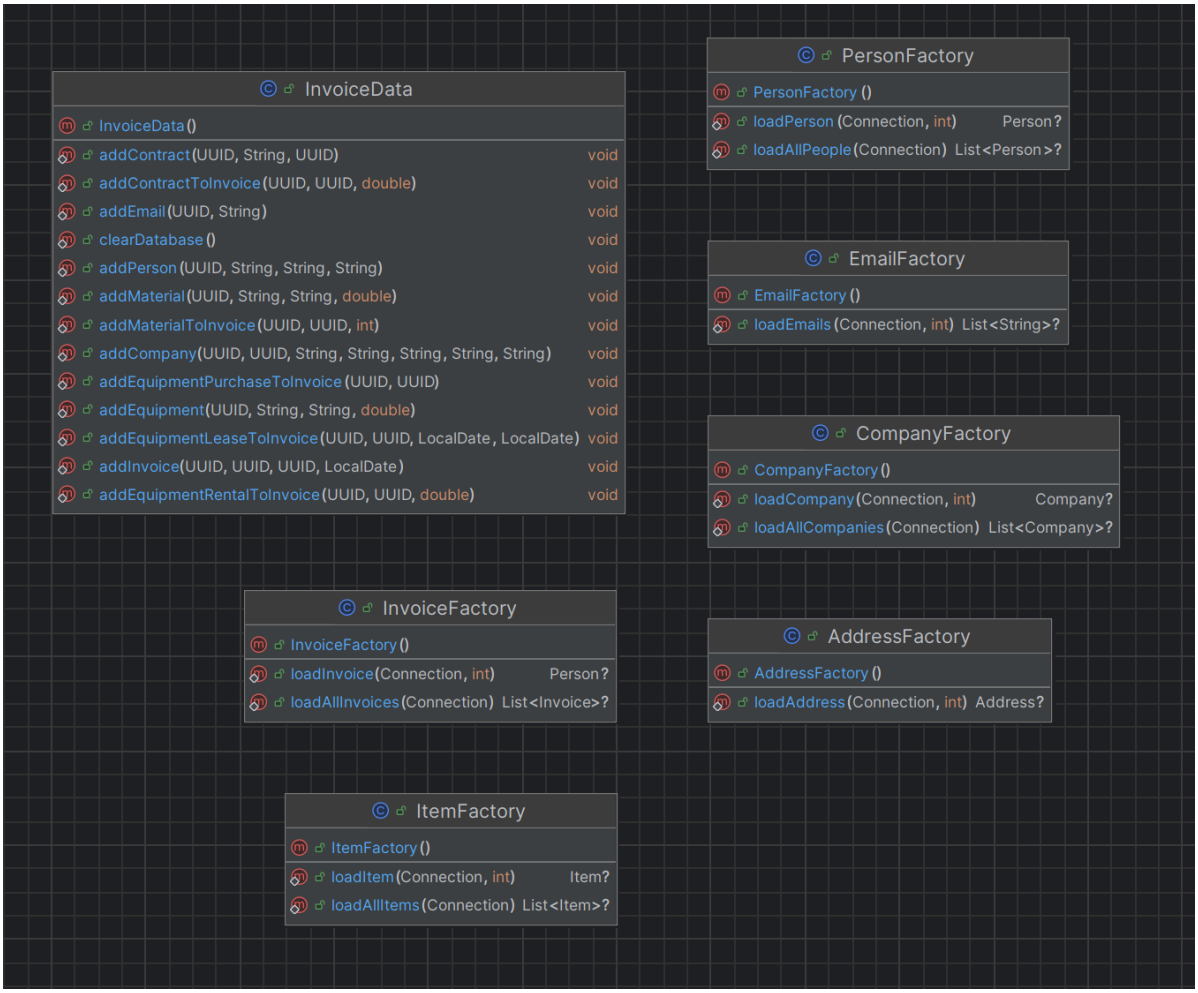

Figure 3: A UML diagram of the remaining classes in the invoice subsystem.

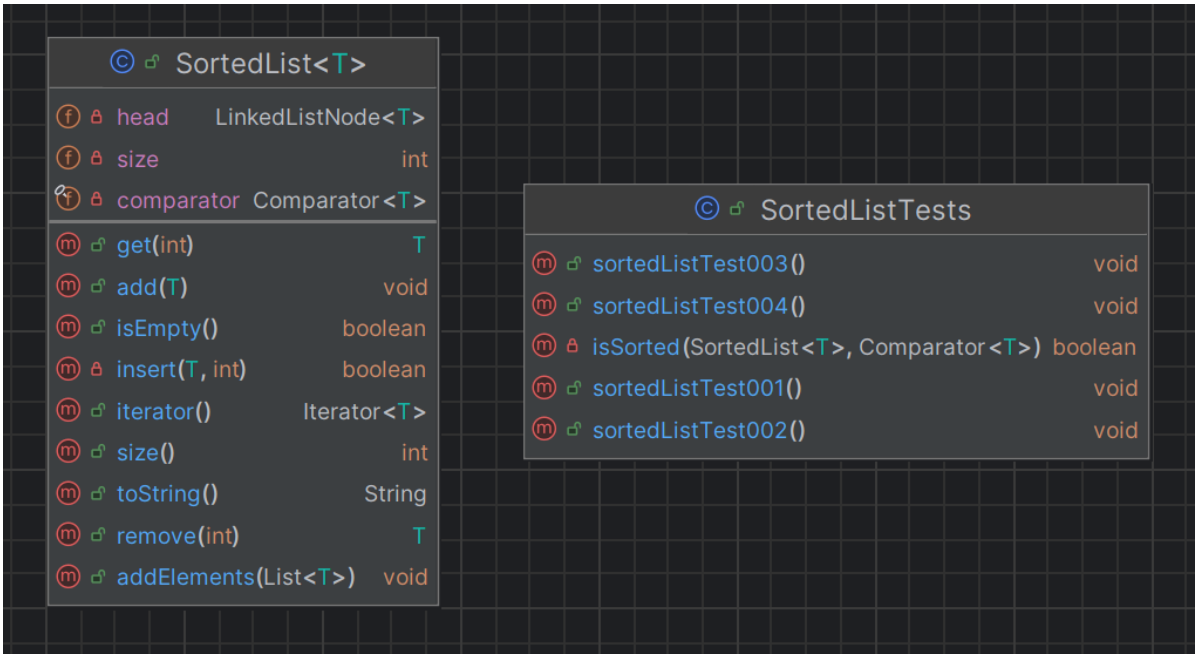

Figure 4: A UML diagram of the sorted list and testing classes in the invoice subsystem.

UML diagrams (see Figure 2, Figure 3, and Figure 4) have been provided to show the layout of all classes and objects involved in the invoice subsystem. Below is a breakdown of each class represented in the above diagram.

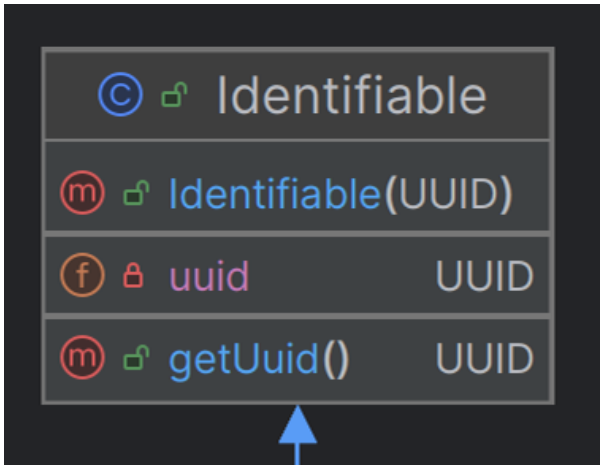

Figure 5: A UML diagram of an Identifiable class.

The Identifiable class (see Figure 5) represents an element that has a unique UUID. The Person, Company, Invoice, and Item inherit this class to contain their UUID information.

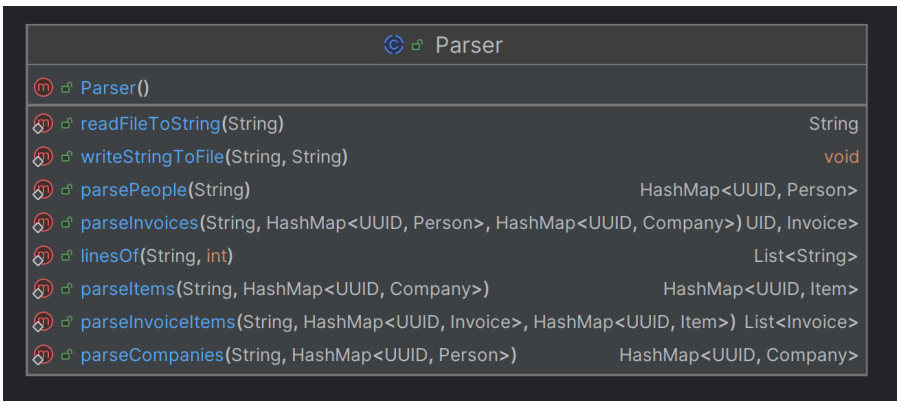

Figure 6: A UML diagram of the Parser class.

The Parser class (see Figure 6) is used to create Person objects, Company objects, and Item subclasses from flat data files. It also provides methods for writing to and reading from files.

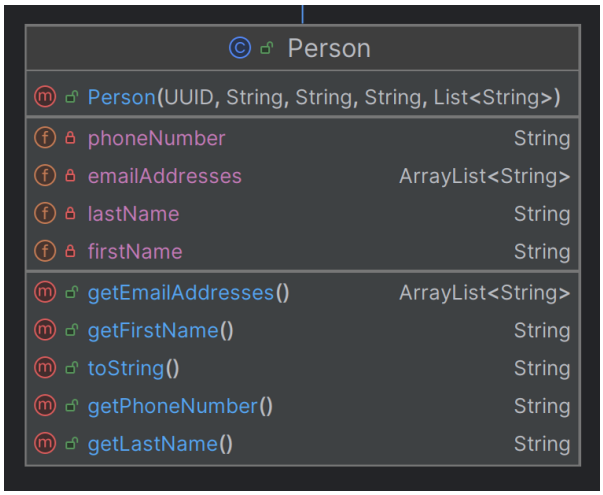

Figure 7: A UML diagram of a Person class.

The Person class (see Figure 7) is a representation of a person in the invoice subsystem, containing information about the UUID, name, phone number, and email addresses of a specific person.

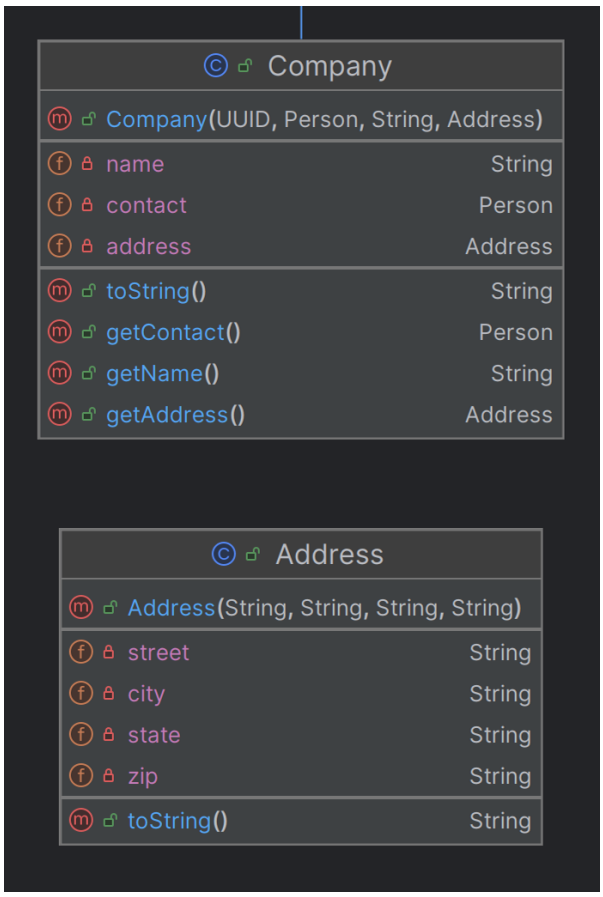

Figure 8: A UML diagram of a Company class and an Address class.

The Company class (see Figure 8) represents a company, containing the UUID, contact UUID (the UUID of a Person class that should be contacted to interface with this company), name, and address of the company. The Address class represents an address, containing the street, city, state, and zip of a specific location. The Address class is separate as to give the Company class less responsibility over address data and to encapsulate String processing methods. The field ‘contactUuid‘’s UUID value should match with a UUID value from a Person class.

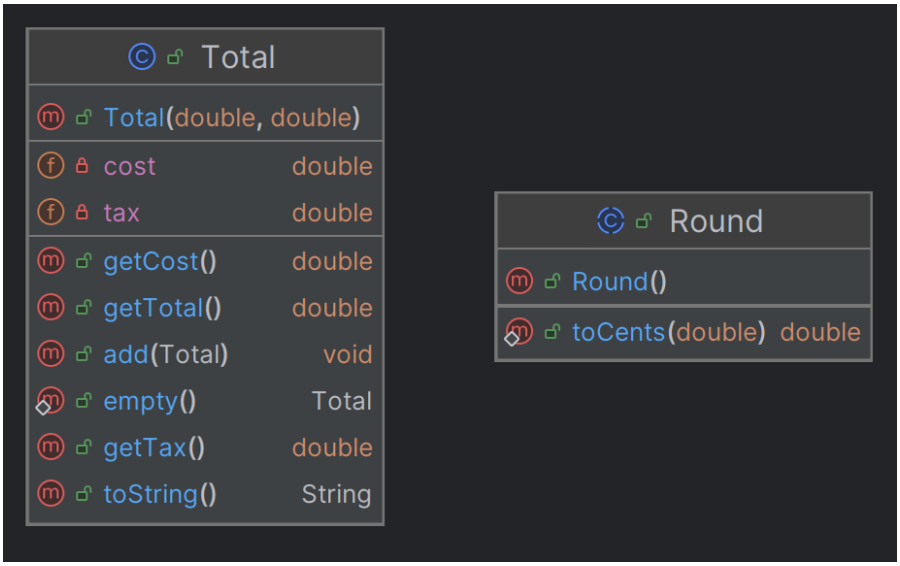

Figure 9: A UML diagram of the Total and Round classes.

The Total class and Round class (see Figure 9) handle financial information. The Total class represents a cost, tax, and total and the Round class provides a method to round money amounts to cents.

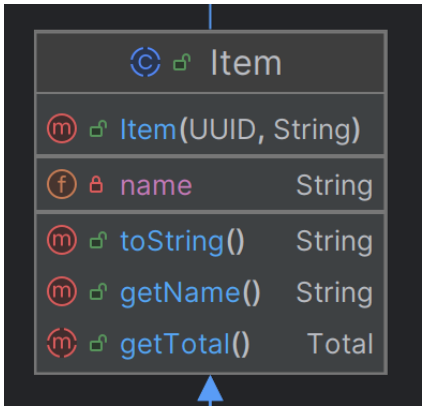

Figure 10: A UML diagram of the Item class.

The Item class (see Figure 10) represents an item that the company handles, each needing a method to return their cost and tax amounts.

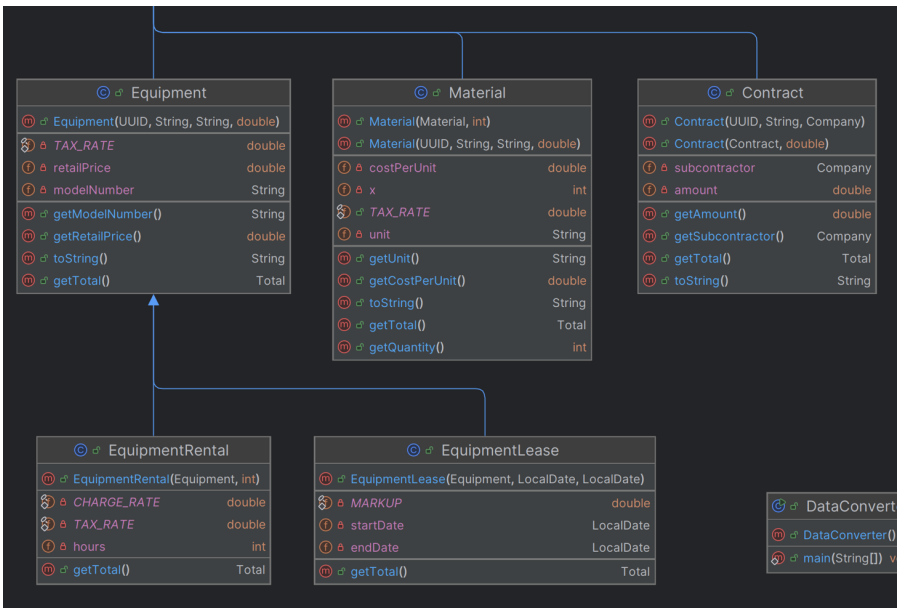

Figure 11: A UML diagram of the Item subclasses.

The Item class has subclasses for construction equipment (Equipment) and its subclasses (EquipmentRental, EquipmentLease), construction materials (Material), and contracts with subcontractors (Contract). The three subclasses Equipment, Material, and Contract all inherit from the Item class, as the Item class contains necessary base data. All implement their own getTotal() method which returns a Total.

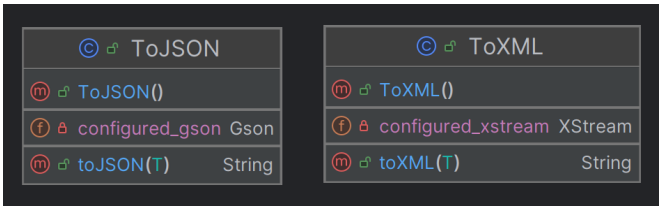

Figure 12: A UML diagram of the ToJSON class and the ToXML class.

The ToJSON and ToXML classes (see Figure 12) are used to export Person, Company, Item, and related classes to JSON and XML formats, respectively. They are able to take a singular class or list of classes and convert them into a JSON or XML string.

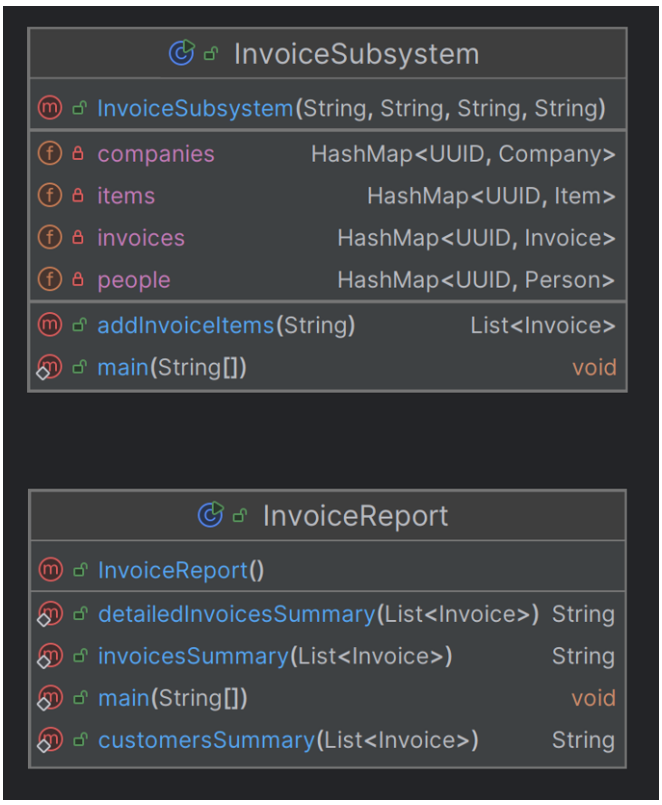

Figure 13: A UML diagram of the InvoiceSubsystem class and the InvoiceReport class.

The InvoiceSubsystem class and InvoiceReport class handle invoices and generating reports. InvoiceSubsystem links all of the information about people, companies, and items and guarantees their integrity. InvoiceReport generates reports from invoices generated by the InvoiceSubsystem.

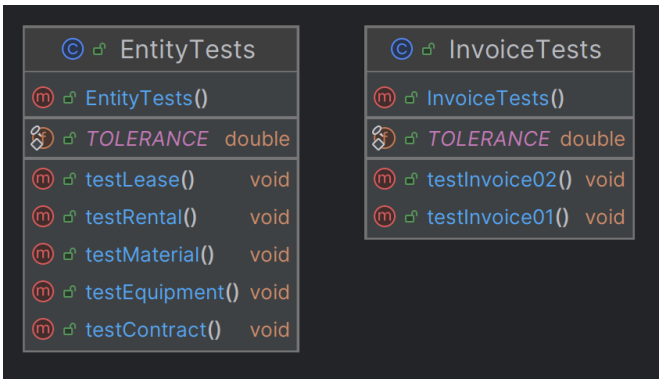

Figure 14: A UML diagram of the EntityTests class and the InvoiceTests class.

The EntityTests and InvoiceTests provide tests for items and invoices, respectively. They are described in more detail in the next section.

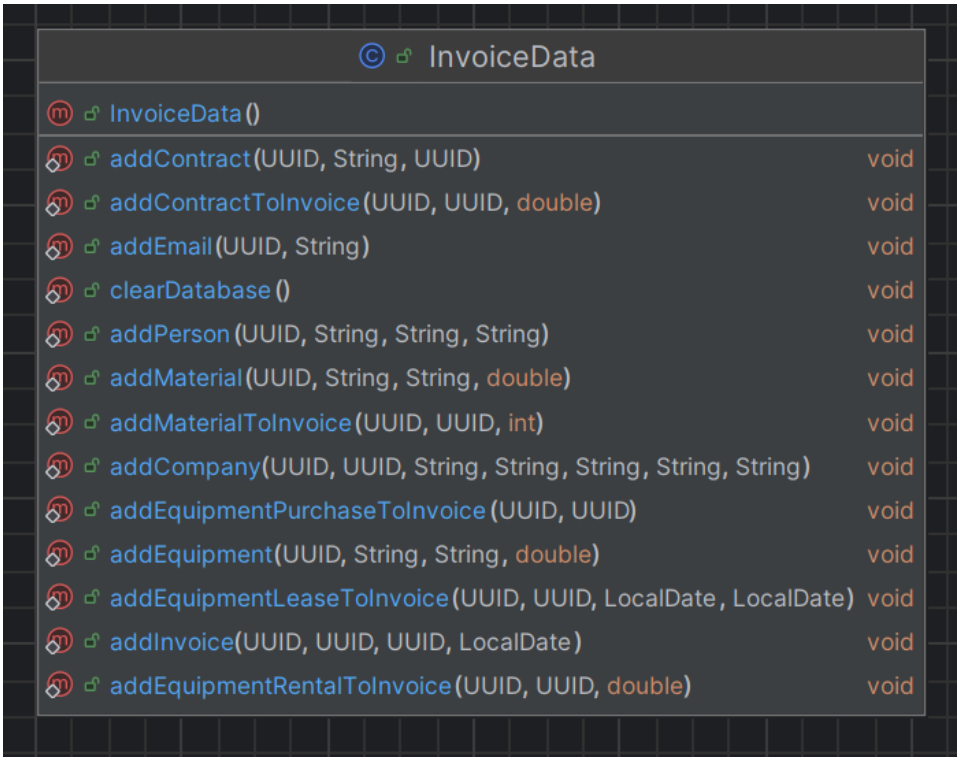

Figure 15: A UML diagram of the InvoiceData class.

The InvoiceData class (see Figure 15) houses most the logic for adding Java objects to the database. The compact nature of the class is to aid in testing (especially with the client-provided tester).

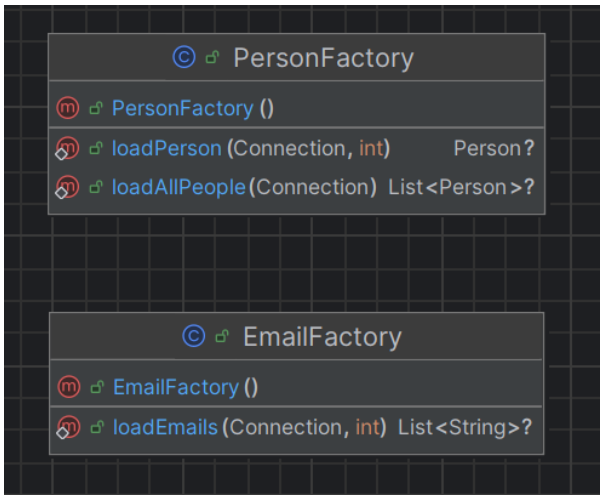

Figure 16: A UML diagram of the PersonFactory and EmailFactory classes.

The PersonFactory and EmailFactory classes (see Figure 16) are used to read the Person

class from the database for use in the rest of the subsystem.

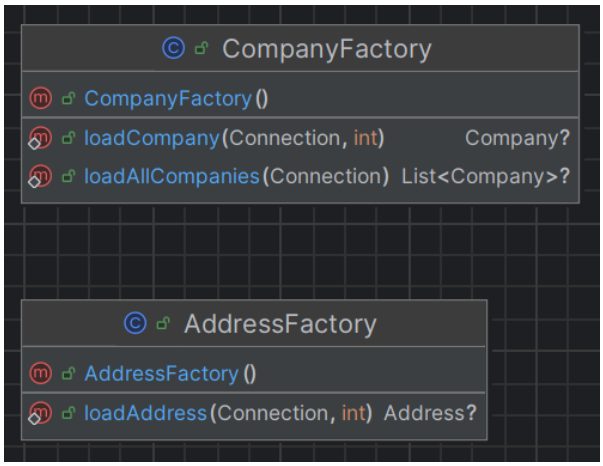

Figure 17: A UML diagram of the CompanyFactory and AddressFactory classes.

The CompanyFactory and AddressFactory classes (see Figure 17) are used to read the Company and Address classes from the database for use in the rest of the subsystem.

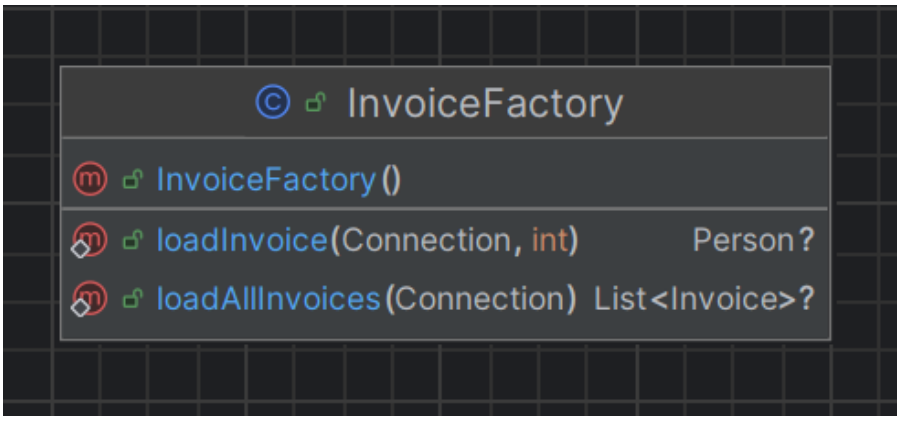

Figure 18: A UML diagram of the InvoiceFactory class.

The InvoiceFactory class (see Figure 18) is used to read the Invoice class from the database for use in the rest of the subsystem.

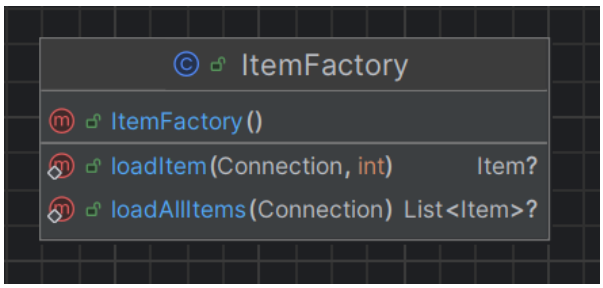

Figure 19: A UML diagram of the ItemFactory class.

The ItemFactory class (see Figure 19) is used to read the Item class from the database for use in the rest of the subsystem.

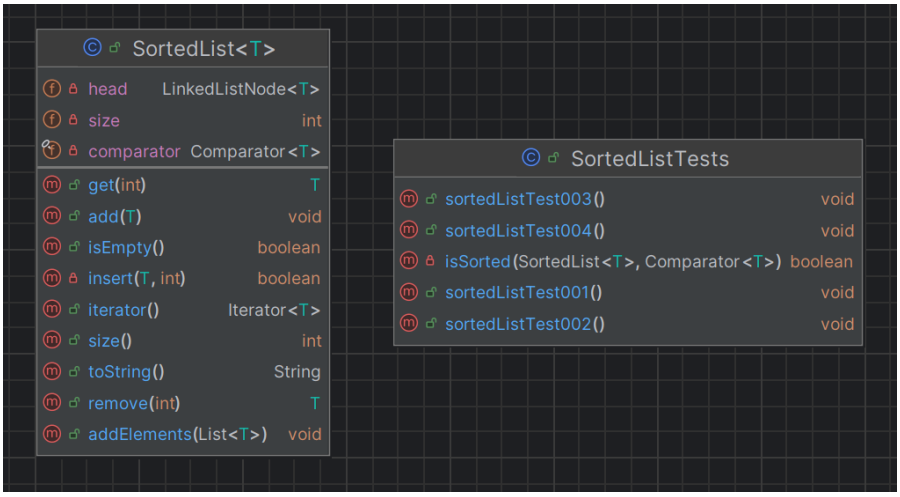

Figure 20: A UML diagram of the class.

The SortedList class (see Figure 20) are used to implement a sorted list data structure for reporting purposes. The SortedList is implemented as a linked list that is always sorted (described in section 3.4).

3.2.1 Component Testing Strategy

The subsystem was tested using a series of unit testing suites focused on different components of the system.

The Parser class was tested across five distinct test sets, each corresponding to a core method. These tests verified that objects were created correctly and that file input/output was handled as expected. Each method was tested against edge cases. For instance, the readFileToString(fileName: String): String method (see Figure 5) was

tested with an empty file (expected to return an empty string), a non-existent file (expected to raise an exception to ensure proper error handling), and a standard plain-text file (expected to return the file's contents).

The object creation methods were tested using sample data sets (approximately five entries) from CSV files. Each resulting object was verified for data integrity and correctness.

Data-storing classes—such as Person, Company, Address, Item, Material, Equipment, and Contract—were tested to ensure they returned correct billing and tax information, with results varying according to the defined tax rules.

The export classes, ToXML and ToJSON, were tested for accurate serialization of both single and multiple objects. For each format, the tests validated that serializing an individual object produced output matching a known valid serialization. Similarly, small lists (around five objects) were serialized and compared to reference outputs to confirm data completeness and structure accuracy.

3.3 Database Interface

The heart of the database interaction logic lives in the InvoiceData.java class. This file functions as the primary API for performing operations that modify the contents of the database. It includes methods for adding new records—such as addPerson(), addCompany(), addInvoice(), and others—that take Java objects as input and persist them into their respective database tables using SQL queries.

In addition to insertion methods, InvoiceData.java also includes several helper functions. These may consist of transaction handling, validation checks, ID lookups, or utility methods for managing foreign key relationships between tables (e.g., linking an invoice to a customer or salesperson). This centralized structure ensures that all data-writing operations follow a consistent logic and that changes to the database schema can be managed from a single point.

Reading data from the database is managed by a series of factory classes, such as 'PersonFactory', 'CompanyFactory', and 'InvoiceFactory'. These classes are responsible for querying the database, interpreting the result sets, and converting the data into Java objects. For instance, 'PersonFactory.getPersonById()' retrieves a row from the 'Person' table and returns a fully initialized 'Person' object. This separation of concerns helps keep the code modular and easier to maintain.

By dividing the read and write responsibilities between 'InvoiceData.java' and the factory classes, the project achieves a clear separation of concerns. This improves code readability, testability, and maintainability.

3.3.1 Component Testing Strategy

The subsystem tested the database infrastructure through a series of unit tests targeting each object in the Java source code and the corresponding database design. Each object was tested using basic operations such as reading from the database and verifying data integrity (ensuring no null values or unintended side effects). Write tests followed a similar approach by inserting objects into the database, retrieving them, and confirming correctness.

Special attention was given to the Item subclasses due to their complex structure in both the Java implementation and the database schema. In general, each object underwent two to three read tests and two to three write tests. The Itemsubclasses were tested more extensively, with up to five read tests per subclass.

Client-provided tests revealed a design issue in the original ItemFactory class. Initially, the factory loaded fully populated Item subclasses (e.g., a Material with quantity or an EquipmentRental with its full data), pulling information from both the Item table and the InvoiceItem table. This approach caused bugs when attempting to add Item subclasses to invoices using methods like InvoiceData.addItemSubclassToInvoice(...), as the required data in the InvoiceItem table had not yet been inserted. To resolve this, the design was modified by splitting the factory into two classes: ItemFactory, which now loads only base item data, and InvoiceItemFactory, which loads items with sales-specific data when it exists in the InvoiceItem table.

Additionally, the structure of the InvoiceData class required each factory class to implement a method like FactoryClass.getId(..., UUID uuid), allowing retrieval of primary keys based on UUIDs. This change preserved the integrity of InvoiceData methods while promoting encapsulation and separation of concerns.

3.4 Design & Integration of a Sorted List Data Structure

For report generation and other purposes, a sorted list data structure is necessary to cleanly handle data and sorting logic. In previous iterations, the report class (InvoiceReport.java) had to manually sort its data, making it difficult to implement different types of reports and read the report generating code.

The sorted list data structure is a node-based list structure that is always sorted. Internally, the sorted list implementation uses a linked list and a comparator (Comparator¡T¿) to impose a total ordering. The list is generalized and can hold different values and types as long as they can be ordered.

For reporting purposes, the sorted list data structure will sort Invoice objects by totals and customer names (alphabetically) and Company objects by the total of all of their invoices. Any ties in sorting will be broken by the UUID of the objects, which will make the comparator slightly more complex but keep generalization for the sorted list data

structure.

The SortedList only has three required operations: adding elements, retrieving elements, and removing elements. There is no way to insert elements because the internal list should always be kept in total order.

The add method simply iterates through the list until it finds a suitable location in the order (i.e. the SortedList's comparator says it is greater than the item before it and less than the item after it). Since the internal implementation is a linked list, the amount of memory used increases with the amount of elements, and no extra allocations or space is needed. The retrieval and removal methods simply the element at the specified index.

3.4.1 Component Testing Strategy

The sorted list data structure was tested using a series of unit tests that populated a test list and verified that its elements were iterated in the correct order. The client-provided testing platform was also used to validate the ordering behavior. Approximately five unit tests were implemented to cover various scenarios, including empty lists and lists containing multiple elements. These tests confirmed that the list correctly handled element insertion, retrieval, and removal operations.

4 Changes & Refactoring

The project had to be refactored many times due to bad encapsulation. Originally, the project was being developed with a more structural paradigm, but later had to be switched to a Object Oriented Programing (OOP) design with proper encapsulation for increased safety and data integrity.

Additionally, the SortedList originally was based off of a custom linked list implementation in a separate class. However, the team decided to merge them together at the end in order to maximize the running time of the sorted list.

5 References

1. **JUnit 5 Framework** – Used for writing and executing unit tests throughout the subsystem.

   https://junit.org/junit5/

2. **MySQL 8.0 Documentation** – Used as the primary relational database management system for storing invoice data.

   https://dev.mysql.com/doc/

References